

Grounded Control Interfaces for Contact-Rich Robot Agents

Feedback Programs, Position–Force Policies, and Joint Residuals

Research Proposal I · Zimo Wen

September 2026

Abstract

Robot agents can describe a goal and invoke manipulation skills while still failing during contact, obstruction, or precise placement. One potential bottleneck is the action interface: a position target expresses where to move but does not fully express how firmly to make contact, when to yield, or how to compensate for local dynamics. This proposal studies whether task-conditioned control representations improve contact-rich manipulation under matched sensing and interaction budgets. Building on Robot-RSI, it compares three approaches: generating executable feedback programs, predicting task-space position–force and impedance parameters, and learning bounded joint-space residual control. The intended paper will establish when a richer control interface improves both task completion and physical interaction quality, and when simpler feedback is sufficient. Experiments separate the contributions of sensing, action representation, control frequency, and training data, with contact quality and deployment cost evaluated alongside success.

1. Motivation, Research Gap, and Paper Hypothesis

The starting point. Robot-RSI already produces executable skills and numerical motion commands. Its LIBERO end-effector increment skill uses servo control and motion solving to produce joint increments and gripper commands. The research question is therefore which additional control variables a model should choose, from which observations, and at what update rate. It is not the introduction of numerical actions into an otherwise purely semantic agent.

The inspected Flexiv adapter exposes joint state, joint torque, external wrench, and gripper-force parameters. A contact-descent handler stops when the measured force change crosses a threshold. These capabilities provide an entry point, but sensing contact and maintaining a desired contact force are different behaviors. The inspected Cartesian-motion wrapper mainly passes pose and velocity and retains hardware-interface checks. Gripper closure force, contact detection, and a six-dimensional end-effector wrench must remain distinct experimental variables.

Motivating tasks. Placing an object on a surface of uncertain height requires a controlled transition from free motion to contact. Pushing along a surface requires maintaining normal contact while progressing tangentially. Pulling a resistant drawer requires distinguishing friction, approach misalignment, and grasp slip. A feedback program, task-space impedance, or joint residual might solve these cases; lower-level output is not assumed to be universally better.

Central hypothesis. A task-conditioned interface that expresses contact goals and feedback can improve manipulation more efficiently than position commands alone, while joint-level learning is useful only when task-space feedback cannot compensate for the relevant error. This is a self-contained paper about **the appropriate action abstraction for robot agents**. Approach B, task-conditioned position–force and impedance control, is the primary method. Feedback programs provide a structured baseline; joint residuals test whether still lower-level output is necessary.

Prior work and intended novelty. Code as Policies already includes feedback programs and impedance-control examples. Diffusion Policy and π_0 provide continuous action-generation foundations. The proposed increment is to ground control structure or parameters in task intent and actual contact evidence, then identify their generalization boundaries. Adding a force field to an API or using a larger action network alone is insufficient. The main claim must be supported by unseen objects and dynamics, matched observations, and measurable physical improvements.

2. Approach A: Executable Local Feedback Programs

Representation and execution. A model generates a program with branches, loops, and termination conditions that reads visual error, proprioception, grasp state, and available force measurements. A program might approach the visible target, reduce speed after contact, and change a local search direction when progress stalls while lateral force rises. The language model supplies inspectable logic; a local runtime executes feedback at a declared rate without requiring a remote language-model call for every small correction.

Generation contract. Expose typed primitives for reading errors, setting short-horizon pose references, observing contact, changing modes, and terminating motion. Each program declares its coordinate frame, action bounds, maximum duration, update period, and exit conditions. Static validation checks types, unbounded loops, and interface consistency. A single control owner applies commands. These checks establish executability, not physical correctness; candidate programs must still be evaluated inside a simulator feedback loop.

Learning and adaptation. Development traces connect scene conditions, candidate programs, and observed responses. The agent generates and revises programs using development feedback and independently verified outcomes. A key comparison asks whether a fixed feedback structure transfers by updating perception-derived parameters, or whether regenerating the entire program is necessary. Test-time models, prompts, and libraries are frozen. Local program generation may occur within an episode, but test outcomes do not become persistent capabilities for later evaluation tasks.

Mechanism test. Compare fixed skills, generated open-loop programs, handcrafted feedback programs, and generated feedback programs. Open- and closed-loop variants share approach trajectories and primitives as far as possible. The difference should concern observation-conditioned corrections rather than access to stronger tools. If the agent only selects a known template, the contribution is task-conditioned selection and transfer, not discovery of a new control law. Measure recovery from local errors, success, and model-call reduction.

3. Approach B: Position–Force and Impedance Policies

Action representation. Predict $a_t = (\Delta x, \Delta R, f^*, K, D, m)$: pose increments, desired contact force, stiffness, damping, and contact mode. A contact frame specifies normal and tangential directions. Selection matrices assign position control and force control to compatible axes, avoiding contradictory constraints on the same degree of freedom. Units, frames, timestamps, and update intervals are explicit in every action packet.

Local realization. A controller uses pose, velocity, and force error to track these references. For example, a normal-direction admittance loop can adjust displacement from $f^* - f_t$ while tangential motion follows a path. The policy updates references at a lower rate than the device-supported feedback loop. Parameter amplitude and slew-rate limits constrain changes. Contact establishment, maintenance, and separation have distinct transitions. Positive stiffness is an engineering constraint, not a proof of overall closed-loop stability.

Training. Collect successful and unsuccessful development interactions from verified control templates and learn the mapping from instruction and observation to control parameters. Compare template-conditioned prediction with a continuous action-generation head. Training can supervise parameter trajectories and execution outcomes, but model selection depends on physical performance on validation tasks. Sensor bias, frame conversion, and filter delay are recorded so that gravity or inertial changes are not automatically mistaken for contact events.

Expected advantage and limitations. This representation uses low-dimensional physical structure and may need less data than learning all joint actions. Its success can nevertheless depend on contact-direction estimates and template coverage. Compare fixed parameters, per-task manual tuning, and observation-conditioned parameters to isolate online adaptation. A model that outputs one constant parameter set has learned parameter selection, not necessarily state-dependent contact control.

4. Approach C: Bounded Joint-Space Residual Control

Formulation. Retain a validated baseline controller and use a small local action network to predict a corrective joint torque from proprioception, force sensing, and task context:

$$\tau_t = \tau_{\text{base}}(s_t) + \text{clip}(\delta\tau_\theta(s_t, g, h_t), -b, b).$$

Residual amplitude, slew rate, and contact-power limits are fixed after validation. The local network executes in the control path; language reasoning provides slower task context. Token-by-token generation of high-frequency torque by a general language model is not required.

Learning and interfaces. In a simulator that actually supports torque commands, establish baseline behavior before learning residual corrections through demonstrations or reinforcement learning. Randomize load, friction, delay, and sensor bias to discourage compensation for a single simulator defect. The actor only receives deployable observations. Hidden dynamics may define evaluation strata or offline analyses but never become actor inputs. Hardware transfer requires checking control frequency, communication delay, and baseline-controller equivalence.

Action Interfaces for Robot Agents

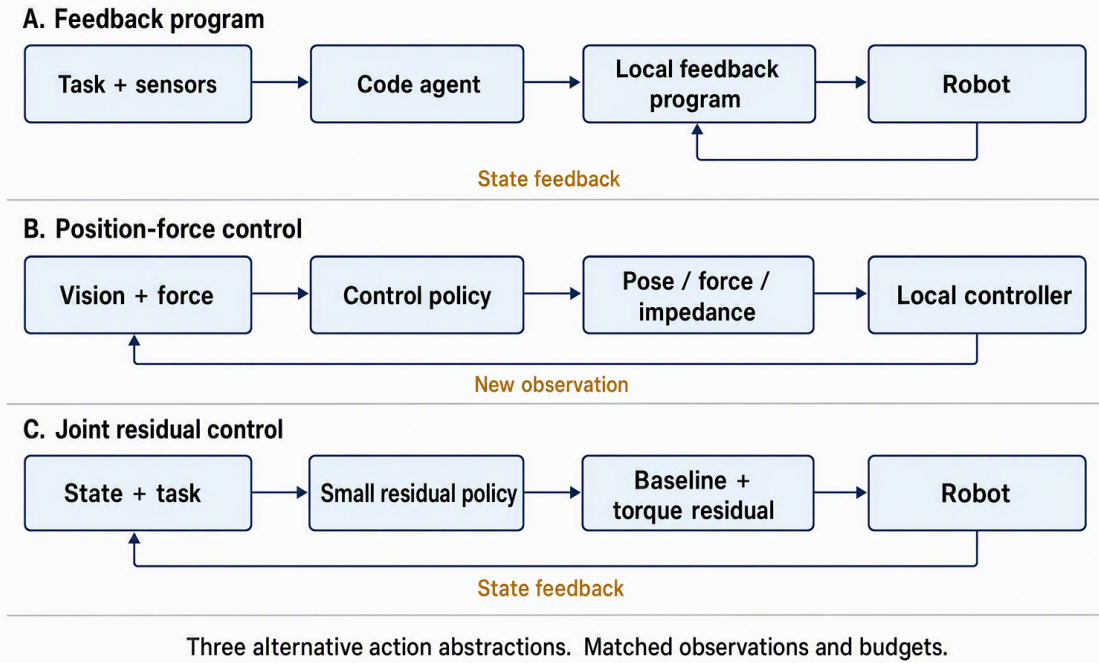


Figure 1: Candidate control interfaces. The three rows produce feedback programs, task-space references, or joint residuals. In row B, robot execution is included in the controller path and observations return from robot sensors. GPT-image-2 conceptual diagram; no measured result is depicted.

Discriminating comparison. Compare no residual, end-effector residuals, impedance adjustment, and joint torque residuals with matched sensing, training data, and approximately matched capacity. If lower-level output helps only for specific disturbances, report that boundary. Retain unstable and saturated trajectories. Clipping a residual constrains execution but does not establish reliability or prove that the learned component caused a successful task.

5. Experimental Design and Evaluation Panels

Standard versus contact evaluation. The standard 120-task LIBERO short panel tests whether the new execution component preserves basic capability. It reports frozen task-level pass@10 by spatial/object/goal subset and task/object/swap/lan suite. A separate contact-control extension evaluates force and torque interfaces, documenting changes to controllers, sensors, and action spaces. Improvements in this extension do not count as standard-benchmark gains.

Main contact panel. Use three task families: contact placement, surface pushing, and drawer interaction. Four object or geometry configurations per family give 12 frozen task instances. Development and validation use different objects, layouts, and physical conditions. Three predetermined disturbance levels and five initial seeds per task yield 180 episodes per method. Each task varies quantities with a clear physical interpretation, such as friction, resistance, or surface height.

First vary friction, load, surface location, and action delay separately; then evaluate held-out combinations, such as surface-height error combined with changed friction. Development verifies that the perturbation range remains solvable using a known effective controller. Greater resistance must not be confused with physical impossibility. Slip and contact-loss criteria use task-appropriate windows and report detection errors.

Comparison	Action and feedback	Matched conditions
Position control	Pose or joint increments	Vision, proprioception, and available force
Contact-stop baseline	Position motion with force threshold	Calibration and execution limits
Handcrafted feedback	Fixed program and tuned controller	Primitives and observation delay
Generated feedback	Conditions and recovery branches	Primitives and model budget
Learned contact policy	Pose, force, and impedance	Training data and contact limits
Joint residual	Baseline plus bounded torque	Observations and approximate capacity

Separating representation from sensing. The main comparison supplies the same force observations to all methods, followed by a no-force ablation. This separates access to better information from the ability to express different control. Record control-update rate, action horizon, and low-level limits. Where architectures cannot be identical, report parameter count, training data, training compute, and deployment latency, with capacity- or latency-matched auxiliary comparisons.

Physical-robot validation. If the Flexiv continuous-force interface and logging chain are verified, use the three matched task families with three physical configurations and ten randomized trials each: an initial 90 trials per method. Calibration and model selection occur before formal trials; training and test objects remain separate. Include joint residuals only if the actual torque interface and controller have been validated. Otherwise, retain torque conclusions as simulation-only and avoid presenting them as successful sim-to-real transfer.

Prespecified endpoints. Final task success is the main behavioral endpoint. Select one task-appropriate contact-quality endpoint before testing, such as peak placement force or force-tracking error during sustained pushing. The panel sizes above are proposed starting points; validation variance determines whether additional task diversity is necessary to support the intended effect-size claim.

6. Planned Experiments and Expected Interpretations

Experiment A: action abstraction. Compare position interfaces, generated feedback, contact parameters, and joint residuals under identical visible observations. Evaluate task success together with interaction quality so that faster completion with a larger impact is not summarized as an unqualified improvement. Retain force and action traces for failed episodes as well as successful ones.

Physical metrics. For contact establishment, report peak force, contact impulse, overshoot, and settling time. During contact maintenance, report force-tracking RMSE, contact-loss fraction, and slip. During motion, report position error, completion time, and abnormal oscillation. Force error is computed only during intervals with a defined target force; including free-space intervals would artificially reduce the average. Report physical units by task before applying any preregistered cross-task aggregation.

Experiment B: contact modes and compliance. Hold the policy structure fixed and remove adaptive stiffness, adaptive damping, mode transitions, or short-term feedback in turn. Compare force-only, compliance-only, and joint prediction. Align parameter traces with contact phases to identify redundant outputs or constant parameter selection. Evaluate whether parameters change appropriately when resistance increases, rather than merely whether the final task succeeds.

Experiment C: whether generated feedback is used. Keep program structure and total action budget fixed while delaying or freezing one feedback channel, or removing the recovery branch relevant to a validated local disturbance. Test whether the program changes behavior after a contact anomaly. Bound full-program regeneration so that results do not rely on unlimited replanning. Attribute failures to perception, conditions, primitives, or interfaces before labeling them language-reasoning failures.

Experiment D: what residuals compensate. Change loads and friction and compare learned residuals with equal-magnitude random residuals, end-effector residuals, and a better-tuned baseline. If the learned residual mainly cancels a constant gravity-compensation error, calibration may be a simpler explanation. Introduce delay and sensor bias to test amplification of high-frequency noise. Report all protective interventions and unstable traces.

Possible result	Defensible interpretation
Feedback improves recovery; contact references reduce force peaks	The interfaces address different errors; explain behavioral and physical gains separately.
Impedance matches torque residuals with less data	Low-dimensional structure suffices; joint-level output is not generally necessary.
Residuals help only under unseen loads	Supports a specific compensation role; examine regression under ordinary conditions.
Success rises alongside oscillation or protection triggers	Overall control quality has not necessarily improved.
Force sensing helps, but action interfaces perform similarly	Sensing may dominate; narrow the representation claim.

Statistics and cost. Use matched seeds and bootstrap intervals clustered by task instance. Randomize hardware method order within trial blocks to reduce thermal and equipment-state effects. Record model calls, tokens, perception/action/recovery time, training volume, and protection-trigger rate. Label secondary analyses explicitly to avoid selecting favorable curves after testing.

7. Feasibility, Risks, and Expected Paper Contribution

An actual interface is a prerequisite. The inspected LIBERO path uses joint increments. Force and torque experiments require implemented, functionally validated controllers. Minimal checks cover free-space tracking, contact with a known surface, coordinate transforms, sensor offsets, mode changes, and command latency. Hardware checks combine SDK semantics with functional trials. These establish that the research variable exists; a working controller is not itself evidence that learning improved behavior.

Stability boundaries. Generated programs, positive impedance parameters, and bounded residuals do not guarantee stability under arbitrary contact. Analyze appropriate local models or passivity assumptions where possible and measure behavior outside those assumptions. Report saturation frequency and the actions actually executed whenever predictions frequently hit limits, allowing readers to distinguish model behavior from the baseline controller’s protective behavior.

Simulation-to-real uncertainty. Friction, contact stiffness, filtering, and tool parameters can differ between simulation and hardware. Document the training randomization range, validation choices, and real test conditions separately. Success with one object does not establish contact generalization. Matched and unseen conditions both require evaluation, and non-transferring failure modes remain part of the result.

Outcome accounting. Only final simulator or independent-harness verdicts count as task success. Infrastructure interruptions, implementation errors, and protection events are reported separately. A method that often fails to execute cannot claim superior reliability from the success rate of its remaining valid episodes. Main tables include scheduled instances, valid verdicts, success/failure counts, infrastructure records, and implementation-error coverage. Preserve videos, force curves, trajectories, diagnostic runs, and exact manifests.

Expected contribution. The paper should establish which errors are addressed by feedback programs, task-space contact references, and joint residuals; introduce a grounded mapping from task intent and contact evidence to executable control; and evaluate that mapping on unseen objects and dynamics. Contact quality, inference delay, and data cost are release criteria alongside success. The contribution is a supported account of when a richer interface is useful, not the fact that an agent can output lower-level numbers.

Independence from the other papers. This study uses a fixed skill library and declared control models. It does not require a new responsiveness algorithm or persistent GPT review. The candidate approaches share a task and logging protocol but receive separate mechanism analyses. The proposed sample counts and anticipated outcomes are experimental designs, not completed results. Training loss remains a diagnostic; trained policies must be tested in simulation on matched and held-out tasks.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [2] Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. 2023. <https://arxiv.org/abs/2303.04137>
- [3] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [4] Octo: An Open-Source Generalist Robot Policy. 2024. <https://arxiv.org/abs/2405.12213>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>